

Tarefa 17: Tipos Derivados em MPI

Produto matriz-vetor com a matriz distribuída por colunas

Professor: Samuel Xavier de Souza

Aluno: Vinícius Silva do Carmo

Setembro de 2026

1. Objetivo

Reimplementar a tarefa 16 ($y = A \cdot x$), agora distribuindo as colunas de A entre os processos. Usar `MPI_Type_vector` e `MPI_Type_create_resized` para definir um tipo derivado que represente colunas da matriz, `MPI_Scatter` com esse tipo para distribuir blocos de colunas e `MPI_Scatter` para enviar os segmentos correspondentes de x . Cada processo calcula uma contribuição parcial para todos os elementos de y , e `MPI_Reduce` com `MPI_SUM` soma os vetores parciais no processo 0. Discutir as diferenças de acesso à memória e de desempenho em relação à distribuição por linhas.

2. Ambiente de Execução

Item	Valor
Máquina	Supercomputador NPAD/UFRN, partição amd-512
Nó	r2n16 — 64 núcleos físicos reservados, de 1 a 64 processos no mesmo nó
Processador	2 × AMD EPYC 7713 (Milan)
MPI	OpenMPI 5.0.10 (libraries/openmpi/5.0.10-gnu14-ucxmt)
Compilador	GCC 14.2.0 via mpicc -O2
Job	SLURM 2135226

O job não usou o nó com exclusividade (a fila para o nó inteiro estava em dias); por isso foi até 64 processos, e outros jobs podiam estar rodando nos demais núcleos. A versão por linhas da tarefa 16 rodou no mesmo job, para a comparação ser feita no mesmo nó.

3. Implementação

A matriz continua armazenada por linhas no processo 0 ($A[i][j] = i + j$), x com todos os valores iguais a 1). Uma coluna é um tipo derivado: `MPI_Type_vector(M, 1, N, MPI_DOUBLE)` descreve M blocos de 1 elemento separados por N elementos. O extent desse tipo vai do primeiro ao último elemento da coluna, então a `MPI_Type_create_resized` o reduz para 1 double: assim, a coluna seguinte começa no elemento seguinte, e o `MPI_Scatter` com contagem N/P entrega N/P colunas consecutivas a cada processo (N deve ser múltiplo de P).

Cada processo recebe as suas colunas como $M \cdot N/P$ doubles contíguos, uma coluna após a outra, e o segmento correspondente de x (N/P valores) por outro `MPI_Scatter`. O cálculo percorre as colunas locais somando $a[:,jj] \cdot x[jj]$ num vetor parcial de tamanho M , e o `MPI_Reduce` com `MPI_SUM` soma esses vetores no processo 0, que confere cada $y[i]$ com o valor exato $N \cdot i + N(N-1)/2$; todas as execuções deram 0 erros.

O processo 0 mede com `MPI_Wtime` a **distribuição** (os dois `MPI_Scatter`), o **cálculo** e a **coleta** (`MPI_Reduce`); na versão por linhas, distribuição é `MPI_Scatter` + `MPI_Bcast` e coleta é `MPI_Gather`. Foram usadas matrizes quadradas de 1024, 4096 e 16384 com 1, 2, 4, ..., 64 processos, 3 repetições de cada caso, ficando a de menor tempo total.

4. Resultados

Tempos em ms; “col.” é a distribuição por colunas (esta tarefa) e “lin.” a por linhas (tarefa 16).

Matriz 1024 × 1024

Proc.	Total col.	Total lin.	Distrib. col.	Distrib. lin.	Cálculo col.	Cálculo lin.	Coleta col.	Coleta lin.
1	8,98	2,16	8,55	1,16	0,43	1,00	0,002	0,002
2	10,3	3,81	10,0	3,28	0,26	0,50	0,015	0,031
4	12,6	5,16	12,4	4,88	0,18	0,26	0,051	0,017
8	11,2	3,37	11,0	3,06	0,15	0,13	0,043	0,17
16	11,4	1,29	11,1	0,94	0,060	0,063	0,17	0,28
32	11,4	1,50	11,2	1,23	0,024	0,032	0,19	0,24
64	12,7	2,03	12,5	1,74	0,018	0,017	0,20	0,28

Matriz 4096 × 4096

Proc.	Total col.	Total lin.	Distrib. col.	Distrib. lin.	Cálculo col.	Cálculo lin.	Coleta col.	Coleta lin.
1	983,1	30,1	974,7	13,4	8,49	16,7	0,007	0,018
2	980,9	33,1	976,3	24,7	4,50	8,40	0,045	0,017
4	975,0	31,2	971,0	26,8	3,58	4,38	0,45	0,040
8	967,1	26,1	962,9	22,0	3,99	4,05	0,22	0,072
16	973,3	16,6	969,4	12,2	3,69	3,83	0,29	0,58
32	975,1	11,1	973,0	9,06	1,82	1,40	0,21	0,64
64	1.214	11,7	1.213	10,9	1,09	0,36	0,11	0,50

Matriz 16384 × 16384

Proc.	Total col.	Total lin.	Distrib. col.	Distrib. lin.	Cálculo col.	Cálculo lin.	Coleta col.	Coleta lin.
1	16.538	474,6	16.409	205,4	128,3	269,3	0,039	0,015
2	16.399	443,7	16.335	308,5	64,6	135,1	0,080	0,039
4	16.335	421,2	16.284	352,8	51,5	68,3	0,12	0,045
8	16.148	345,2	16.094	291,5	53,7	52,4	0,32	1,28
16	16.091	226,2	16.041	174,9	48,9	49,0	2,09	2,27
32	16.158	144,0	16.131	117,4	25,6	25,8	1,70	0,81
64	19.392	112,3	19.377	97,2	12,5	13,8	2,45	1,20

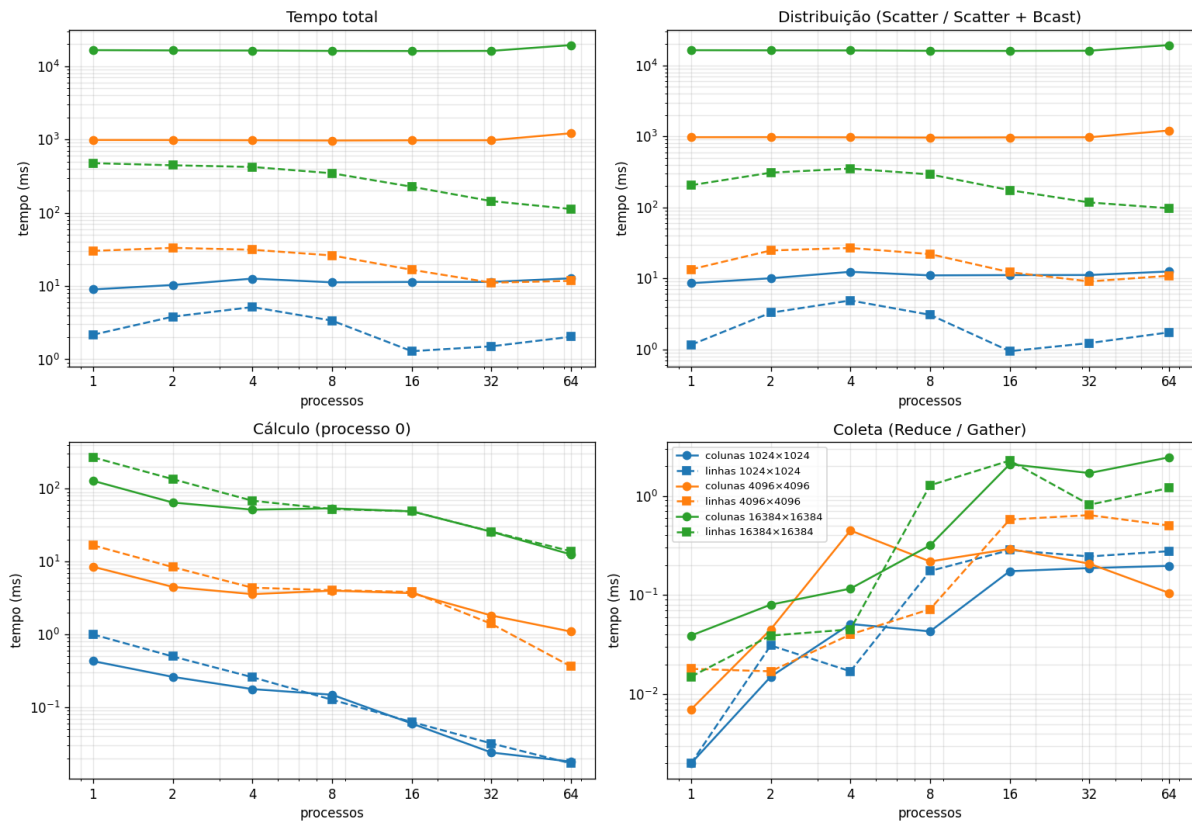


Figura 1 — Tempo total, distribuição, cálculo e coleta em função do número de processos. Linhas cheias: distribuição por colunas; tracejadas: por linhas.

5. Análise

Distribuir colunas de uma matriz armazenada por linhas é muito mais caro. Na matriz de 16384×16384 (2 GB), o MPI_Scatter por colunas leva 16,4 s com 1 processo, contra 205,4 ms por linhas (80× mais lento), e com 64 processos a diferença chega a 199×. Na de 4096×4096 a razão fica entre 36 e 111×, e na de 1024×1024 entre 3 e 12×. A distribuição passa a ser praticamente todo o tempo, e o tempo total não cai com mais processos.

O motivo é o acesso à memória. Por linhas, o bloco de cada processo é um trecho contíguo de A e o MPI_Scatter é uma cópia sequencial, que aproveita cada linha de cache inteira. Por colunas, os elementos de uma coluna estão separados por N doubles (128 KB na maior matriz): o MPI empacota elemento por elemento, e cada leitura cai numa linha de cache (e, na maior matriz, numa página de memória) diferente, da qual só 8 dos 64 bytes são usados naquele momento. O tipo derivado evita escrever esse laço no programa, mas o laço continua existindo dentro do MPI. Na maior matriz, isso dá uns 131 MB/s de empacotamento, contra uns 10,5 GB/s da cópia contígua. Além disso, o empacotamento é feito pelo processo 0 para todos os destinos, por isso não fica mais rápido com mais processos (por linhas, as cópias contíguas acontecem em paralelo), e com 64 processos ainda piorou.

O cálculo local ficou até mais rápido. Como cada processo recebe as colunas já contíguas, o laço percorre a memória em ordem nas duas versões, e o tempo de cálculo é praticamente o mesmo a partir de 8 processos. Com poucos processos, a versão por colunas chega a ser 2,1× mais rápida: ela atualiza $y[i] += a \cdot x[j]$ com iterações independentes, que o compilador consegue vetorizar, enquanto o produto escalar da versão por linhas acumula tudo numa única soma, que sem `-ffast-math` não pode ser reordenada.

Troca de x inteiro por y inteiro. Por colunas, cada processo recebe só N/P valores de x, mas guarda um y parcial de M valores, e o MPI_Reduce combina P vetores de tamanho M, enquanto o MPI_Gather junta só M valores no total. Mesmo assim a coleta fica em poucos milissegundos (2,45 ms contra 1,20 ms na maior matriz com 64 processos) e é irrelevante perto da distribuição.

6. Conclusão

Com `MPI_Type_vector` e `MPI_Type_create_resized` foi possível distribuir colunas com um único `MPI_Scatter`, como se fossem blocos contíguos, e o `MPI_Reduce` com `MPI_SUM` montou o resultado correto. O desempenho, porém, é muito pior que o da distribuição por linhas: a matriz está armazenada por linhas, e ler colunas dela é um acesso espaçado, que desperdiça a cache e é feito só pelo processo 0. O cálculo e a coleta são parecidos nas duas versões; o que decide é a distribuição. A divisão por colunas só compensaria se a matriz já estivesse armazenada por colunas, ou se cada processo gerasse as suas próprias colunas.

7. Código-Fonte

```
1  #include <mpi.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  int main(int argc, char **argv) {
6      int rank, size;
7      MPI_Init(&argc, &argv);
8      MPI_Comm_rank(MPI_COMM_WORLD, &rank);
9      MPI_Comm_size(MPI_COMM_WORLD, &size);
10
11     int m = atoi(argv[1]), n = atoi(argv[2]);
12     int colunas = n / size; /* colunas por processo */
13
14     /* uma coluna: m doubles separados por n; extent de 1 double para que a
15      * próxima coluna comece logo no elemento seguinte */
16     MPI_Datatype col, coluna;
17     MPI_Type_vector(m, 1, n, MPI_DOUBLE, &col);
18     MPI_Type_create_resized(col, 0, sizeof(double), &coluna);
19     MPI_Type_commit(&coluna);
20
21     double *A = NULL, *x = NULL, *y = NULL;
22     double *a = malloc((size_t)m * colunas * sizeof(double)); /* colunas locais */
23     double *x1 = malloc(colunas * sizeof(double));
24     double *y1 = calloc(m, sizeof(double)); /* contribuição parcial */
25
26     if (rank == 0) { /* A[i][j] = i + j, x = 1 */
27         A = malloc((size_t)m * n * sizeof(double));
28         x = malloc(n * sizeof(double));
29         y = malloc(m * sizeof(double));
30         for (int i = 0; i < m; i++)
31             for (int j = 0; j < n; j++) A[(size_t)i * n + j] = i + j;
32         for (int j = 0; j < n; j++) x[j] = 1.0;
33     }
34
35     double t0 = MPI_Wtime();
36     /* cada processo recebe as suas colunas uma após a outra: a[j*m + i] */
37     MPI_Scatter(A, colunas, coluna, a, m * colunas, MPI_DOUBLE, 0, MPI_COMM_WORLD);
38     MPI_Scatter(x, colunas, MPI_DOUBLE, x1, colunas, MPI_DOUBLE, 0, MPI_COMM_WORLD);
39     double t1 = MPI_Wtime();
40
41     for (int j = 0; j < colunas; j++)
42         for (int i = 0; i < m; i++) y1[i] += a[(size_t)j * m + i] * x1[j];
43     double t2 = MPI_Wtime();
44
45     MPI_Reduce(y1, y, m, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
46     double t3 = MPI_Wtime();
47
48     if (rank == 0) { /* y[i] = n*i + n(n-1)/2 */
49         int erros = 0;
50         for (int i = 0; i < m; i++)
51             if (y[i] != (double)n * i + (double)n * (n - 1) / 2) erros++;
52         printf("%d,%d,%d,%.6f,%.6f,%.6f,%.6f,%d\n", m, n, size,
53             t3 - t0, t1 - t0, t2 - t1, t3 - t2, erros);
54         free(A);
55         free(x);
56         free(y);
57     }
58     free(a);
59     free(x1);
60     free(y1);
61     MPI_Finalize();
62     return 0;
63 }
64
```

Figura 2 — mxv.c.